

W3_hw1 Report

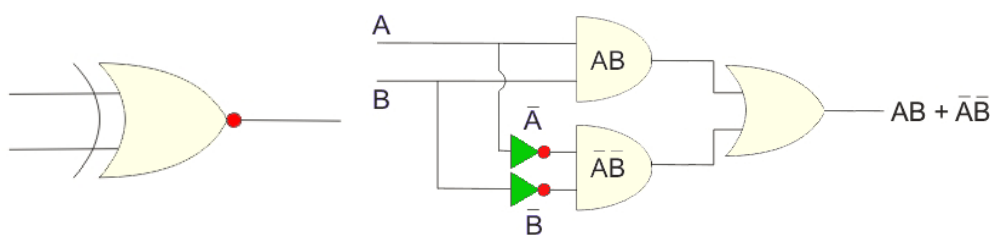
[1]Design and Implement XNOR Gate

1. Objectives:

- (1) Dataflow, Behavioral, Gate-Level Modeling Method가 무엇인지 이해하고 원리를 파악한다.
- (2) XNOR Gate를 세 가지 방법으로 디자인할 수 있다.
- (3) Module을 실행시키기 위한 Testbench가 어떤 방식으로 작동하는지 파악하고 직접 디자인할 수 있다.

2. Theoretical Approach:

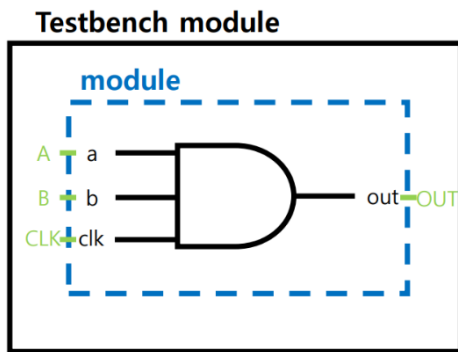
Dataflow Modeling은 Gate의 원형이 어떻게 생겼는지 알 수 있는 모델링으로 Boolean expression을 이용한다. XNOR Gate는 $AB + \bar{A}\bar{B}$ 이므로 이를 이용해 디자인했다. Behavioral Modeling은 Gate가 정확히 어떤 메커니즘으로 작동하는지 그 행동을 서술한다. 주로 if ~ else if ~ else 구문이 사용되는데 해당 실험에서도 이를 이용했다. Gate-Level Modeling은 모델링하고자 하는 대상을 AND, OR, NOT Gates로 모두 바꾸어 input/output을 설정하는 방식이다.



Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

왼쪽 상단의 그림과 같이 나타낼 수 있는 XNOR Gate를 오른쪽 상단의 그림과 같이 모두 AND, OR, NOT Gates로 분해하고 중간에 연결된 Gates의 outputs을 wire로 연결하여 input/output을 입력/출력하는 방법이다. 왼쪽의 그림은 XNOR Gate에서 inputs A, B에 따라 달라지는 output X 값을 나타낸 Truth Table이다.

Testbench는 module을 실행시키기 위해 필수적으로 필요한 module로 그 안에 module(s),



module과 testbench사이의 interface(port), parameters, clock initializer, test patterns를 포함하고 있다. 왼쪽의 그림은 Testbench module과 그것이 포함하고 있는 세부적인 elements들간의 관계를 도식화하여 나타낸 것이다.

3. Verilog Implementation:

(1) Dataflow Modeling of XNOR Gate :

```
module xnor_dataflow_gate (a, b, out);

    input a, b;

    output out;

    assign out = (a && b) || (!a && !b);

endmodule
```

XNOR Gate의 Boolean Expression은 $AB + \overline{A}\overline{B}$ 이다. Logic Operator로 표현하면 and는 '&&', or는 '||', not은 '!'이므로 이를 이용해 Boolean Expression을 그대로 Logic Operators로 변환하여 out을 할당한 것이다.

(2) Behavioral Modeling of XNOR Gate :

```
module xnor_behavioral_gate (a, b, out);

    input a, b;

    output out;

    reg out;

    always @ (a or b)

    begin

        if (a == 1'b0 && b == 1'b0)

            out <= 1'b1;

        else if (a == 1'b1 && b == 1'b1)
```

```

        out <= 1'b1;

    else

        out <= 1'b0;

    end

endmodule

```

Theoretical Approach에 첨부한 Truth Table에 의하면 input a, b가 둘 다 0이거나 1일 때 output은 1이 나오고 그 외 나머지 두 경우는 output이 0이 나온다. 이를 if ~ else 구문으로 표현하여 data가 나오는 모습을 행동적으로 나타낸 것이다.

(3) Gate-Level Modeling of XNOR Gate :

```

module xnor_gatelevel_gate (a, b, out);

    input a, b;

    output out;

    wire not_a_out, not_b_out;

    wire and_1_out, and_2_out;

    not_gate not_1 (.a(a), .out(not_a_out));

    not_gate not_2 (.a(b), .out(not_b_out));

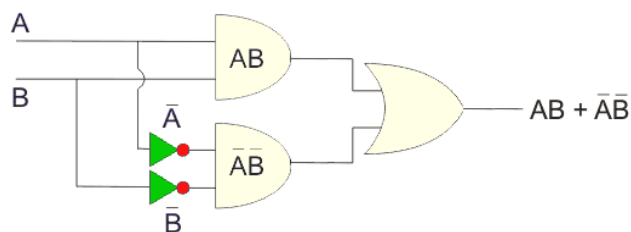
    and_gate and_1 (.a(a), .b(b), .out(and_1_out));

    and_gate and_2 (.a(not_a_out), .b(not_b_out), .out(and_2_out));

    or_gate or_1 (.a(and_1_out), .b(and_2_out), .out(out));

endmodule

```



wire는 중간에 위치한 gates의 outputs를 연결한 것인데 2개의 not gates의 outputs를 각각 not_a_out, not_b_out, 2개의 and gates의 outputs를 각각 and_1_out, and_2_out으로 선언한 것이다. not_1에서는 input으로 a를 받아서 not_a_out을 도출했고 not_2에서는 b를 받아서 not_b_out을 도출했다.

and_1에서는 a와 b를 받아서 둘의 and 값인 and_1_out을 도출했고 and_2에서는 not a와 not b를 받아 and_2_out을 도출했다. 마지막으로 or_1에서 and_1_out과 and_2_out을 받아 최종 out을 도출해냈다.

(4) Testbench of XNOR Gate :

```
module tb_w3_hw1;

    reg A, B;

    wire OUT_NOR_D, OUT_NOR_G, OUT_NOR_B;

    wire OUT_XNOR_D, OUT_XNOR_G, OUT_XNOR_B;

    nor_dataflow_gate nor_dataflow_module(.a(A), .b(B), .out(OUT_NOR_D));

    nor_behavioral_gate nor_behavioral_module(.a(A), .b(B), .out(OUT_NOR_B));

    nor_gatelevel_gate nor_gatelevel_module(.a(A), .b(B), .out(OUT_NOR_G));

    xnor_dataflow_gate xnor_dataflow_module(.a(A), .b(B), .out(OUT_XNOR_D));

    xnor_behavioral_gate xnor_behavioral_module(.a(A), .b(B), .out(OUT_XNOR_B));

    xnor_gatelevel_gate xnor_gatelevel_module(.a(A), .b(B), .out(OUT_XNOR_G));

    initial

    begin

        A = 0; B = 0;

        #10 A = 0; B = 0;

        #10 A = 0; B = 1;

        #10 A = 1; B = 0;

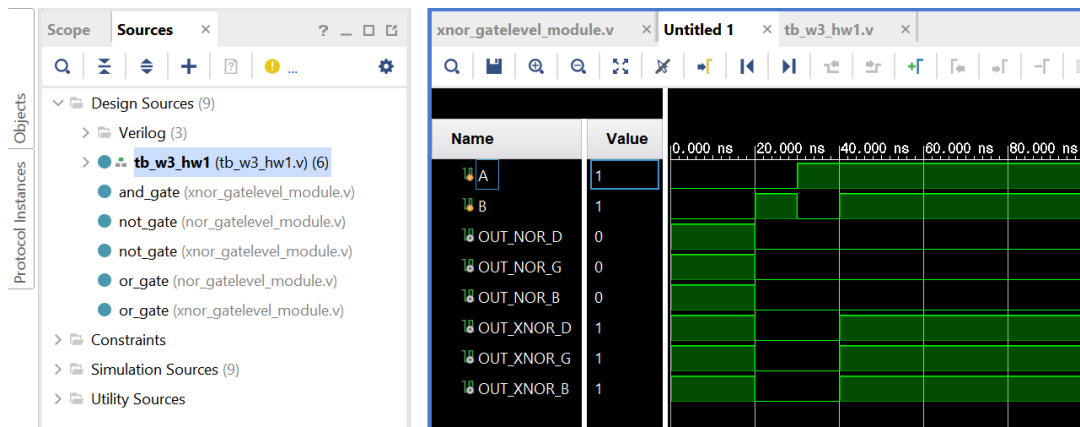
        #10 A = 1; B = 1;

    end

endmodule
```

예시로 주어진 NOR Gate에 대한 ports 외에 추가적으로 HW에 해당하는 XNOR Gate에 대한 ports를 reg와 wire로 연결하였다. 그리고 test pattern을 주었는데 처음에 A, B를 모두 0으로 설정하고 10ns 후에도 계속 0으로 유지되다가 또 10ns 후에, 즉 20ns 후에는 A = 0, B = 1로 설정되고 다음 10ns 후에는 A = 1, B = 0으로 설정되었다가 10ns 이후부터는 계속 A = 1, B = 1로 유지되도록 값을 설정하였다.

4. Simulation Results:



Simulation 결과를 살펴보면 Testbench에서 준 값이 정확히 반영되었음을 알 수 있다. 20ns까지 A, B 모두 0이고 10ns 후까지 A = 0, B = 1로 유지되다가 다음 10ns 후까지는 A = 1, B = 0, 이후부터는 A, B 모두 1로 유지되고 있음을 알 수 있다. XNOR의 결과 또한 Truth Table을 정확히 반영하고 있는데 A, B가 모두 1이거나 0일 때 output이 1로 나오고 A, B가 다르면 output이 0으로 나오는데 이 결과가 simulation에 그대로 나타나고 있음을 알 수 있다.

5. Conclusion:

XNOR Gate를 Dataflow, Behavioral, Gate-Level Modeling의 방법으로 source code를 작성하고 Testbench module까지 완성하여 실제로 simulation을 해보았다. 그 결과 Truth Table에서 확인한 input 값에 따른 output이 정확히 도출됨을 직접 확인하였다. 결과가 예상한 대로 도출되었기 때문에 세 가지 모델링을 활용하여 할당한 out에 대한 source code가 정확했음을 알 수 있다.